ESTR3114 Project Resource Guide

Gradient Conflict in Joint Token Prediction and Sentiment Classification
Compiled 10 November 2026 | 31 resources | Click a title or URL to open

01 / Research papers and theory

Read PCGrad first, then the multi-objective paper. The last two entries are optional background.

01. Gradient Surgery for Multi-Task Learning

Yu et al., 2020. Main paper for PCGrad: conflicting gradients, projection, algorithm and experiments.
https://arxiv.org/abs/2001.06782

02. PCGrad: official code

Authors' TensorFlow implementation. Use as an algorithm reference for the project's PyTorch implementation.
https://github.com/tianheyu927/PCGrad

03. Multi-Task Learning as Multi-Objective Optimization

Sener and Koltun, 2018. Background on Pareto optimality and competing task objectives.
https://arxiv.org/abs/1810.04650

04. Multi-objective optimization: authors' code

Reference implementation accompanying Sener and Koltun. Implementing this alternative method is optional.
https://github.com/isl-org/MultiObjectiveOptimization

05. Decoupled Weight Decay Regularization

Loshchilov and Hutter. AdamW paper explaining decoupled weight decay.
https://arxiv.org/abs/1711.05101

06. GradNorm [optional]

Chen et al., 2018. Adaptive loss balancing through gradient magnitudes; useful for an extension or discussion.
https://proceedings.mlr.press/v80/chen18a.html

07. Convex Optimization [optional]

Boyd and Vandenberghe. Free book and slides for optimization background. GPT-2 training is non-convex.
https://web.stanford.edu/~boyd/cvxbook/

02 / Dataset and pretrained model

Use the TweetEval sentiment configuration and the openai-community/gpt2 checkpoint.

08. TweetEval dataset

Download the sentiment configuration with its train, validation and test splits.

https://huggingface.co/datasets/cardiffnlp/tweet_eval

09. TweetEval: official repository

Label mappings, preprocessing conventions, benchmark evaluation and dataset updates.

https://github.com/cardiffnlp/tweeteval

10. TweetEval benchmark paper

Barbieri et al., 2020. Dataset background and the benchmark citation for the report.

https://aclanthology.org/2020.findings-emnlp.148/

11. GPT-2 Small checkpoint and model card

Pretrained weights and matching tokenizer used by the shared Transformer backbone.

https://huggingface.co/openai-community/gpt2

12. Original GPT-2 repository

Original model background and link to Language Models are Unsupervised Multitask Learners.

https://github.com/openai/gpt-2

13. Hugging Face Datasets: loading

How to load the dataset and select configurations, revisions and splits.

https://huggingface.co/docs/datasets/en/loading

14. GPT-2 API documentation

Model classes, hidden states, token outputs and sequence classification behavior.

https://huggingface.co/docs/transformers/en/model_doc/gpt2

15. GPT-2 source code

Inspect backbone/head structure and classification pooling. The main branch may change over time.

https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling_gpt2.py

03 / Training and gradient implementation

Build ordinary joint training first, then add separate gradient extraction and PCGrad.

16. PyTorch installation selector

Select a suitable installation for the operating system and GPU environment.

https://pytorch.org/get-started/locally/

17. Transformers installation

Set up the library used to load GPT-2 and its tokenizer.

https://huggingface.co/docs/transformers/en/installation

18. Causal language-modeling tutorial

Next-token training, labels and collation. Preserve individual tweet labels when adapting the example.

https://huggingface.co/docs/transformers/en/tasks/language_modeling

19. Padding and truncation

Prepare variable-length tweets and attention masks; exclude padding from the task computations.

https://huggingface.co/docs/transformers/main/pad_truncation

20. torch.autograd.grad

Extract each task's gradients separately. Returned gradients are not accumulated into parameter .grad fields.

https://docs.pytorch.org/docs/stable/generated/torch.autograd.grad.html

21. torch.optim.AdamW

Optimizer API for ordinary joint updates and updates using PCGrad-modified gradients.

https://docs.pytorch.org/docs/stable/generated/torch.optim.AdamW.html

22. Gradient cosine similarity

Measure alignment or conflict between task gradients globally and by layer.

https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.cosine_similarity.html

23. Gradient vector norm

Measure gradient magnitudes and the ratio of token-task to sentiment-task gradient norms.

https://docs.pytorch.org/docs/stable/generated/torch.linalg.vector_norm.html

04 / Evaluation and experiment tools

Use identical evaluation procedures across conditions and record the software versions for reproducibility.

24. Perplexity of fixed-length models

Language-model evaluation, token-loss aggregation and context-window considerations.

https://huggingface.co/docs/transformers/en/perplexity

25. Sentiment macro-F1

Use average="macro" for the unweighted mean F1 across the three sentiment classes.

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html

26. Sentiment accuracy

Compute the fraction of correctly classified texts.

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html

27. PyTorch reproducibility

Random seeds, deterministic execution options and reproducibility limitations.

https://docs.pytorch.org/docs/stable/notes/randomness.html

28. TensorBoard with PyTorch

Track losses, sentiment scores, gradient statistics and convergence curves.

https://docs.pytorch.org/docs/stable/tensorboard.html

29. Matplotlib quick start

Create convergence plots, layer-wise conflict figures and objective trade-off plots.

https://matplotlib.org/stable/users/explain/quick_start.html

30. CUDA synchronization

Reference for synchronizing GPU work when measuring elapsed training time.

https://docs.pytorch.org/docs/stable/generated/torch.cuda.synchronize.html

31. Google Colab [optional]

Alternative notebook environment if local execution is unsuitable. Check resource availability before planning runs.

https://colab.research.google.com/